BSE4104: Emerging Trends in Software Engineering
8-Week AI-Native and Agentic Engineering Capstone

WEEK 5 WEEKLY PROGRESS REPORT: AGENT ARCHITECTURE AND BOUNDED AUTONOMY

Group: Group G
Project: University Student-Support Case Agent
Reporting period: 28 September – 2 October 2026

1. Work Completed Against Weekly Objectives

Week 5 turned the tool-calling work of Week 4 into one goal-directed, multi-step workflow with explicit limits. The task chosen is following up an existing support case: the agent looks the case up, reads the result, and only then decides whether a follow-up ticket is justified. It benefits from multi-step decision making because the second action depends on what the first tool returns.

The Sense/Context → Plan/Decide → Act/Tool → Observe → Stop/Re-plan loop was implemented by direct orchestration (no framework) in src/bounded_agent.py, using Gemini function calling. The model proposes the next action; the application decides whether it is allowed. The boundaries are:

- Maximum of 3 iterations; the run stops with a message if the limit is reached.
- Approved tools only: get_case_status and create_support_ticket. Any other tool name is rejected.
- One tool call at a time; if the model requests several at once, the run stops.
- Stop condition: the model requests no further tool, or a limit or rejection is reached.
- Ticket creation is authorised by application code, not by the model. It requires an explicit student request (negations such as "do not create a ticket" and process questions do not count), a successful lookup of the same case, a case status of in progress or under review, a valid category, a non-empty description, and no earlier ticket in the same run.
- Grades, fees, admissions and official records cannot be changed because no tool exists for them and the system instruction forbids it.

Three execution traces were captured in docs/evidence/week5_execution_traces.json:

No.	Scenario	Observed behaviour
1	Check CASE-1045 and create a follow-up ticket	get_case_status, then create_support_ticket (TICKET-1001), then stop. Three iterations; response reports only what the tools confirmed.
2	Check CASE-9999 (does not exist)	get_case_status returned "No case found"; no ticket was attempted; the agent asked the student to verify the case ID and stopped.
3	Request to change a grade	No tool was called; the agent declined and explained what it can do.

The workflow is covered by 14 automated unit tests that run without an API key (Ran 14 tests, OK; output in docs/evidence/bounded_agent_unit_test_results.txt). They cover the authorisation rules above, the iteration limit and an end-to-end lookup-then-ticket run.

2. Key Engineering Decisions

- **Application-side authorisation:** a tool call from the model is a request, not permission. Each call is checked against the rules above before anything runs.
- **Direct orchestration:** a plain loop was chosen over a framework so every step can be traced, limited and tested.

- **Fail closed:** an unknown tool, missing argument, unknown case or failed check returns an error and no action is taken.
- **Simulated side effect:** tickets are created in a simulated store and case data is synthetic, so no real university record is touched.
- **Human approval:** in this version the student's explicit request is the approval for the low-risk simulated ticket. There is no staff approval step; higher-impact actions are simply not available to the agent.

3. Challenges and Responses

Testing exposed gaps in the first version of the authorisation logic, which were corrected and are now protected by regression tests: a ticket could rely on an earlier successful lookup after a later lookup failed; a ticket could be raised for a different case from the one checked; resolved cases were not rejected; an empty description could be turned into the text "None"; and a negated request could be read as a request. Each now has its own test.

The Week 5 Agent Failure and Safety Review (13 deterministic tool and authorisation tests) was written against the earlier agent.py design, so its results describe tools.py and the allow-list logic but should be re-run against bounded_agent.py for the final submission.

Trace 2 shows a safe failure (stop and ask the student to verify the case ID) rather than a retry within the same run. A recovery would take place in a follow-up message with a corrected ID; this was covered by tests but not captured as a separate live trace.

4. Individual Contributions and Evidence

Member	Responsibility	Evidence to retain
Stacey Ampaire	Bounded agent workflow, execution traces, authorisation fixes and unit tests	src/bounded_agent.py; tests/test_bounded_agent.py; docs/evidence/ (traces and test results)
Judith Kemirembe	Agent Task Contract; tool functions and validation (get_case_status, create_support_ticket)	Agent Task Contract (submitted on ClickUp); src/tools.py
Desmond Cole Aineamatsiko	Agent failure and safety review	docs/evaluation/ Week5_Agent_Failure_Safety_Review.docx
Arnold Fred Kawuma	Agent Architecture Diagram	Agent Architecture Diagram (submitted on ClickUp)
Rachel Mirembe	Documentation and evidence coordination	Repository evidence and ClickUp task updates

5. Items to Add to the Repository

- Agent Architecture Diagram (Arnold Fred Kawuma) and Agent Task Contract (Judith Kemirembe) were submitted on ClickUp and are not yet committed to the repository. They will be added under docs/architecture/ and docs/requirements/ respectively.